Trade Event Streaming (WebSocket-first) - Implementation Report
Member G2-M5: Complete Code Documentation

TABLE OF CONTENTS

1. [WebSocket Aggregator Gateway](#1-websocket-aggregator-gateway)
2. [Trade Service - Event Publishing](#2-trade-service---event-publishing)
3. [Trade-Driven Pricing Service](#3-trade-driven-pricing-service)
4. [Telemetry Service](#4-telemetry-service---observability)
5. [Frontend Market Data Service](#5-frontend-market-data-service)
6. [Frontend Order Service](#6-frontend-order-service)

---

1. WebSocket Aggregator Gateway

Purpose
The WebSocket Aggregator serves as the central gateway for real-time event streaming. It:
- Maintains WebSocket connections from multiple clients
- Manages client subscriptions to different event channels (trades, orders, market data, options prices)
- Implements the **G2-M5 Trade Replay Strategy**: sends last N trades per symbol on client reconnect
- Batches messages efficiently (50ms interval) to reduce network overhead
- Broadcasts trade executions, market data, options prices, and positions to subscribed clients
- Records WebSocket metrics for telemetry/monitoring

Key Features
- **Trade Replay on Reconnect**: Sends configurable number of historical trades (default 50, max 200)
- **Multi-channel Subscriptions**: Clients can subscribe to TRADES, ORDERS, MARKET_DATA, POSITIONS, ORDER_BOOK, OPTIONS_PRICE
- **Message Batching**: 50ms aggregation window to batch messages before sending
- **Telemetry Integration**: Records connection events, in/out message counts

Code

```java
package com.helesto.socket;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Collection;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

import javax.enterprise.context.ApplicationScoped;
import javax.inject.Inject;
import javax.websocket.OnClose;
import javax.websocket.OnError;
import javax.websocket.OnMessage;
import javax.websocket.OnOpen;
import javax.websocket.Session;
import javax.websocket.server.ServerEndpoint;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.helesto.model.OrderEntity;
import com.helesto.model.TradeEntity;
import com.helesto.service.OrderBookManager;
import com.helesto.service.ReferenceDataService;
import com.helesto.service.TelemetryService;
import com.helesto.service.TradeService;

/**
 * G4-M1: WebSocket Aggregator Gateway
 * - Single WS endpoint aggregating order states, trades, market data,
positions
 * - Client subscription management
```

```
 * - Efficient broadcast with batching
 */
@ServerEndpoint("/ws/aggregator")
@ApplicationScoped
public class WebSocketAggregator {

    private static final Logger LOG =
LoggerFactory.getLogger(WebSocketAggregator.class);
    private static final ObjectMapper mapper = new ObjectMapper();

    // Connected sessions
    private final Set<Session> sessions =
ConcurrentHashMap.newKeySet();

    // Subscriptions per session
    private final Map<Session, ClientSubscription> subscriptions = new
ConcurrentHashMap<>();

    // Message batching
    private final ScheduledExecutorService scheduler =
Executors.newSingleThreadScheduledExecutor();
    private final Map<Session, List<Message>> pendingMessages = new
ConcurrentHashMap<>();
    private static final long BATCH_INTERVAL_MS = 50; // 50ms batching

    // G2-M5: Trade replay configuration
    private static final int TRADE_REPLAY_COUNT = 50; // Number of
trades to replay on reconnect

    @Inject
    ReferenceDataService referenceDataService;

    @Inject
    TradeService tradeService;

    @Inject
    TelemetryService telemetryService;

    public WebSocketAggregator() {
        // Start batch sender
        scheduler.scheduleAtFixedRate(this::flushPendingMessages,
                BATCH_INTERVAL_MS, BATCH_INTERVAL_MS,
TimeUnit.MILLISECONDS);
```

```
    }

    @OnOpen
    public void onOpen(Session session) {
        sessions.add(session);
        subscriptions.put(session, new ClientSubscription());
        pendingMessages.put(session, new CopyOnWriteArrayList<>());
        LOG.info("WebSocket client connected: {}", session.getId());

        // Record telemetry
        if (telemetryService != null) {
            telemetryService.recordWsConnection(true);
        }

        // Send welcome message with replay capability notification
        sendDirect(session, new Message("CONNECTED", Map.of(
            "sessionId", session.getId(),
            "tradeReplayAvailable", true,
            "tradeReplayCount", TRADE_REPLAY_COUNT,
            "timestamp", System.currentTimeMillis()
        )));
    }

    @OnClose
    public void onClose(Session session) {
        sessions.remove(session);
        subscriptions.remove(session);
        pendingMessages.remove(session);
        LOG.info("WebSocket client disconnected: {}", session.getId());

        // Record telemetry
        if (telemetryService != null) {
            telemetryService.recordWsConnection(false);
        }
    }

    @OnError
    public void onError(Session session, Throwable error) {
        LOG.error("WebSocket error for session {}: {}",
session.getId(), error.getMessage());
    }

    @OnMessage
```

```
    public void onMessage(String message, Session session) {
        // Record telemetry
        if (telemetryService != null) {
            telemetryService.recordWsMessageIn();
        }

        try {
            Map<String, Object> request = mapper.readValue(message,
Map.class);
            String action = (String) request.get("action");

            switch (action) {
                case "SUBSCRIBE":
                    handleSubscribe(session, request);
                    break;
                case "UNSUBSCRIBE":
                    handleUnsubscribe(session, request);
                    break;
                case "GET_SNAPSHOT":
                    handleGetSnapshot(session, request);
                    break;
                case "PING":
                    sendDirect(session, new Message("PONG", Map.of(
                        "timestamp", System.currentTimeMillis()
                    )));
                    break;
                case "REPLAY_TRADES":
                    handleTradeReplay(session, request);
                    break;
                default:
                    LOG.warn("Unknown action: {}", action);
            }
        } catch (Exception e) {
            LOG.error("Error processing message: {}", e.getMessage());
            sendDirect(session, new Message("ERROR", Map.of(
                "message", e.getMessage()
            )));
        }
    }

    private void handleSubscribe(Session session, Map<String, Object>
request) {
        ClientSubscription sub = subscriptions.get(session);
```

```
        if (sub == null) return;

        String channel = (String) request.get("channel");
        List<String> symbols = (List<String>) request.get("symbols");

        switch (channel) {
            case "ORDERS":
                sub.subscribedOrders = true;
                if (symbols != null) sub.orderSymbols.addAll(symbols);
                break;
            case "TRADES":
                sub.subscribedTrades = true;
                if (symbols != null) sub.tradeSymbols.addAll(symbols);
                break;
            case "MARKET_DATA":
                sub.subscribedMarketData = true;
                if (symbols != null)
sub.marketDataSymbols.addAll(symbols);
                break;
            case "POSITIONS":
                sub.subscribedPositions = true;
                break;
            case "ORDER_BOOK":
                sub.subscribedOrderBook = true;
                if (symbols != null)
sub.orderBookSymbols.addAll(symbols);
                break;
            case "OPTIONS_PRICE":
                sub.subscribedOptionsPrice = true;
                if (symbols != null)
sub.optionsPriceSymbols.addAll(symbols);
                break;
            case "ALL":
                sub.subscribedOrders = true;
                sub.subscribedTrades = true;
                sub.subscribedMarketData = true;
                sub.subscribedPositions = true;
                sub.subscribedOptionsPrice = true;
                break;
        }

        sendDirect(session, new Message("SUBSCRIBED", Map.of(
            "channel", channel,
```

```
            "symbols", symbols != null ? symbols : "all"
        )));

        LOG.info("Session {} subscribed to {}", session.getId(),
channel);
    }

    /**
     * G2-M5: Handle trade replay request on reconnect
     * - Sends last N trades for specified symbols or all symbols
     * - Supports custom count from client request
     */
    private void handleTradeReplay(Session session, Map<String, Object>
request) {
        Object countObj = request.get("count");
        int count = countObj != null ? ((Number) countObj).intValue() :
TRADE_REPLAY_COUNT;
        count = Math.min(count, 200); // Cap at 200 to prevent abuse

        List<String> symbols = (List<String>) request.get("symbols");

        LOG.info("Trade replay requested by session {} - count: {},
symbols: {}",
                session.getId(), count, symbols);

        List<TradeEntity> tradesToReplay;

        if (symbols == null || symbols.isEmpty()) {
            // Replay all recent trades
            tradesToReplay = tradeService.getRecentTrades(count);
        } else {
            // Replay trades for specific symbols
            tradesToReplay = new ArrayList<>();
            int perSymbolCount = Math.max(1, count / symbols.size());

            for (String symbol : symbols) {
                List<TradeEntity> symbolTrades =
tradeService.getTradesBySymbol(symbol);
                tradesToReplay.addAll(symbolTrades.stream()
                    .limit(perSymbolCount)
                    .collect(java.util.stream.Collectors.toList()));
            }
```

```
            // Sort all by timestamp descending and limit
            tradesToReplay.sort((a, b) -> {
                java.time.LocalDateTime aTime = a.getCreatedAt();
                java.time.LocalDateTime bTime = b.getCreatedAt();
                if (aTime == null && bTime == null) return 0;
                if (aTime == null) return 1;
                if (bTime == null) return -1;
                return bTime.compareTo(aTime);
            });
            if (tradesToReplay.size() > count) {
                tradesToReplay = tradesToReplay.subList(0, count);
            }
        }

        // Send replay header
        sendDirect(session, new Message("TRADE_REPLAY_START", Map.of(
            "count", tradesToReplay.size(),
            "timestamp", System.currentTimeMillis()
        )));

        // Send each trade
        for (TradeEntity trade : tradesToReplay) {
            long createdAtMs = trade.getCreatedAt() != null
                ?
trade.getCreatedAt().atZone(java.time.ZoneId.systemDefault()).toInstant
().toEpochMilli()
                : 0;
            sendDirect(session, new Message("TRADE_REPLAY", Map.of(
                "tradeId", trade.getTradeId(),
                "symbol", trade.getSymbol(),
                "price", trade.getPrice(),
                "quantity", trade.getQuantity(),
                "buyOrderId", trade.getBuyOrderId(),
                "sellOrderId", trade.getSellOrderId(),
                "aggressorSide", trade.getAggressorSide(),
                "tradeDate", trade.getTradeDate(),
                "tradeTime", createdAtMs,
                "isReplay", true
            )));
        }

        // Send replay complete
        sendDirect(session, new Message("TRADE_REPLAY_END", Map.of(
```

```
            "count", tradesToReplay.size(),
            "timestamp", System.currentTimeMillis()
        )));

        LOG.info("Trade replay completed for session {} - sent {}
trades",
                session.getId(), tradesToReplay.size());
    }

    /**
     * Broadcast trade execution to subscribed clients
     */
    public void broadcastTrade(TradeEntity trade) {
        Message msg = new Message("TRADE", Map.of(
            "tradeId", trade.getTradeId(),
            "symbol", trade.getSymbol(),
            "price", trade.getPrice(),
            "quantity", trade.getQuantity(),
            "buyOrderId", trade.getBuyOrderId(),
            "sellOrderId", trade.getSellOrderId(),
            "aggressorSide", trade.getAggressorSide(),
            "timestamp", System.currentTimeMillis()
        ));

        for (Session session : sessions) {
            ClientSubscription sub = subscriptions.get(session);
            if (sub != null && sub.subscribedTrades) {
                if (sub.tradeSymbols.isEmpty() ||
sub.tradeSymbols.contains(trade.getSymbol())) {
                    queueMessage(session, msg);
                }
            }
        }
    }

    /**
     * G3-M4: Broadcast options price update to subscribed clients
     * - Triggered when underlying price changes from trade execution
     * - Contains fair price, Greeks, and IV
     */
    public void broadcastOptionsPrice(String symbol, String optionType,
                                      double spotPrice, double
strikePrice,
```

```
                                        double fairPrice, double delta,
double gamma,
                                        double theta, double vega, double
rho,
                                        double impliedVolatility, double
timeToExpiry) {
        Map<String, Object> data = new LinkedHashMap<>();
        data.put("symbol", symbol);
        data.put("optionType", optionType);
        data.put("spotPrice", spotPrice);
        data.put("strikePrice", strikePrice);
        data.put("fairPrice", fairPrice);
        data.put("delta", delta);
        data.put("gamma", gamma);
        data.put("theta", theta);
        data.put("vega", vega);
        data.put("rho", rho);
        data.put("impliedVolatility", impliedVolatility);
        data.put("timeToExpiry", timeToExpiry);
        data.put("timestamp", System.currentTimeMillis());

        Message msg = new Message("OPTIONS_PRICE", data);

        for (Session session : sessions) {
            ClientSubscription sub = subscriptions.get(session);
            if (sub != null && sub.subscribedOptionsPrice) {
                if (sub.optionsPriceSymbols.isEmpty() ||
sub.optionsPriceSymbols.contains(symbol)) {
                    queueMessage(session, msg);
                }
            }
        }
    }

    private void queueMessage(Session session, Message message) {
        List<Message> queue = pendingMessages.get(session);
        if (queue != null) {
            queue.add(message);
        }
    }

    private void flushPendingMessages() {
```

```
        for (Map.Entry<Session, List<Message>> entry :
pendingMessages.entrySet()) {
            Session session = entry.getKey();
            List<Message> messages = entry.getValue();

            if (!messages.isEmpty() && session.isOpen()) {
                // Batch messages
                List<Message> batch = new ArrayList<>(messages);
                messages.clear();

                try {
                    if (batch.size() == 1) {

session.getBasicRemote().sendText(mapper.writeValueAsString(batch.get(0
)));
                    } else {

session.getBasicRemote().sendText(mapper.writeValueAsString(
                            new Message("BATCH", Map.of("messages",
batch))
                        ));
                    }
                } catch (IOException e) {
                    LOG.error("Error sending batch to session {}: {}",
session.getId(), e.getMessage());
                }
            }
        }
    }

    private void sendDirect(Session session, Message message) {
        if (session.isOpen()) {
            try {

session.getBasicRemote().sendText(mapper.writeValueAsString(message));
                if (telemetryService != null) {
                    telemetryService.recordWsMessageOut();
                }
            } catch (IOException e) {
                LOG.error("Error sending to session {}: {}",
session.getId(), e.getMessage());
            }
        }
```

```
    }

    public int getConnectedClients() {
        return sessions.size();
    }

    // Inner Classes
    private static class ClientSubscription {
        boolean subscribedOrders = false;
        boolean subscribedTrades = false;
        boolean subscribedMarketData = false;
        boolean subscribedPositions = false;
        boolean subscribedOrderBook = false;
        boolean subscribedOptionsPrice = false;

        Set<String> orderSymbols = ConcurrentHashMap.newKeySet();
        Set<String> tradeSymbols = ConcurrentHashMap.newKeySet();
        Set<String> marketDataSymbols = ConcurrentHashMap.newKeySet();
        Set<String> orderBookSymbols = ConcurrentHashMap.newKeySet();
        Set<String> optionsPriceSymbols =
ConcurrentHashMap.newKeySet();
    }

    public static class Message {
        public String type;
        public Object data;
        public long timestamp;

        public Message() {}

        public Message(String type, Object data) {
            this.type = type;
            this.data = data;
            this.timestamp = System.currentTimeMillis();
        }
    }
}
```

---

2. Trade Service - Event Publishing

Purpose
The Trade Service is responsible for:
- Persisting executed trades to the database
- Publishing **TradeExecuted events** to registered listeners via observer pattern
- Providing query methods to retrieve trades (by symbol, client, date range, etc.)
- Supporting the replay strategy by providing recent trades

Key Features
- **Event Publishing**: Uses observer pattern with `List<Consumer<TradeEntity>> tradeListeners`
- **Real-time Notifications**: Calls `notifyTradeListeners()` immediately after trade creation
- **Flexible Queries**: Methods to get trades by symbol, client, date range, or recent trades
- **Trade Statistics**: Computes VWAP, high/low, volume for a symbol

Code

```java
package com.helesto.service;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.Consumer;

import javax.enterprise.context.ApplicationScoped;
import javax.inject.Inject;
import javax.persistence.EntityManager;
import javax.transaction.Transactional;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.helesto.model.TradeEntity;

/**
 * G2-M4: Trade Service
 * - Trade persistence and retrieval
 * - Trade event publishing
```

```
 * - Trade statistics
 */
@ApplicationScoped
public class TradeService {

    private static final Logger LOG =
LoggerFactory.getLogger(TradeService.class);
    private static final DateTimeFormatter DATE_FORMAT =
DateTimeFormatter.ofPattern("yyyy-MM-dd");

    @Inject
    EntityManager entityManager;

    // Trade listeners for real-time updates
    private final List<Consumer<TradeEntity>> tradeListeners = new
CopyOnWriteArrayList<>();

    /**
     * Create a trade from match result
     */
    @Transactional
    public TradeEntity createTrade(MatchingEngine.Fill fill,
                                   String incomingOrderId, String
incomingClOrdId,
                                   String incomingClientId, String
incomingSide,
                                   String symbol) {
        TradeEntity trade = new TradeEntity();
        trade.setTradeId(fill.tradeId);
        trade.setExecId(fill.execId);
        trade.setSymbol(symbol);
        trade.setPrice(fill.price);
        trade.setQuantity(fill.quantity);

        // Set buy/sell sides based on incoming order side
        if ("BUY".equals(incomingSide) || "1".equals(incomingSide)) {
            trade.setBuyOrderId(incomingOrderId);
            trade.setBuyClOrdId(incomingClOrdId);
            trade.setBuyClientId(incomingClientId);
            trade.setSellOrderId(fill.contraOrderId);
            trade.setSellClientId(fill.contraClientId);
            trade.setAggressorSide("BUY");
        } else {
```

```
            trade.setSellOrderId(incomingOrderId);
            trade.setSellClOrdId(incomingClOrdId);
            trade.setSellClientId(incomingClientId);
            trade.setBuyOrderId(fill.contraOrderId);
            trade.setBuyClientId(fill.contraClientId);
            trade.setAggressorSide("SELL");
        }

        trade.setTradeDate(LocalDate.now().format(DATE_FORMAT));

trade.setSettlementDate(LocalDate.now().plusDays(2).format(DATE_FORMAT)
); // T+2
        trade.setTradeStatus("CONFIRMED");
        trade.setTradeType("REGULAR");

        entityManager.persist(trade);
        LOG.info("Created trade: {} {} {} @ {}", trade.getTradeId(),
symbol, fill.quantity, fill.price);

        // Notify listeners
        notifyTradeListeners(trade);

        return trade;
    }

    /**
     * Get trade by ID
     */
    public TradeEntity getTradeById(String tradeId) {
        return entityManager.createQuery(
            "SELECT t FROM TradeEntity t WHERE t.tradeId = :tradeId",
TradeEntity.class)
            .setParameter("tradeId", tradeId)
            .getResultStream()
            .findFirst()
            .orElse(null);
    }

    /**
     * Get trades by symbol
     */
    public List<TradeEntity> getTradesBySymbol(String symbol) {
        return entityManager.createQuery(
```

```
            "SELECT t FROM TradeEntity t WHERE t.symbol = :symbol ORDER
BY t.createdAt DESC",
            TradeEntity.class)
            .setParameter("symbol", symbol)
            .getResultList();
    }

    /**
     * Get trades by client
     */
    public List<TradeEntity> getTradesByClient(String clientId) {
        return entityManager.createQuery(
            "SELECT t FROM TradeEntity t WHERE t.buyClientId =
:clientId OR t.sellClientId = :clientId " +
            "ORDER BY t.createdAt DESC", TradeEntity.class)
            .setParameter("clientId", clientId)
            .getResultList();
    }

    /**
     * Get recent trades
     */
    public List<TradeEntity> getRecentTrades(int limit) {
        return entityManager.createQuery(
            "SELECT t FROM TradeEntity t ORDER BY t.createdAt DESC",
TradeEntity.class)
            .setMaxResults(limit)
            .getResultList();
    }

    /**
     * Get trades for a date range
     */
    public List<TradeEntity> getTradesByDateRange(String startDate,
String endDate) {
        return entityManager.createQuery(
            "SELECT t FROM TradeEntity t WHERE t.tradeDate >=
:startDate AND t.tradeDate <= :endDate " +
            "ORDER BY t.createdAt DESC", TradeEntity.class)
            .setParameter("startDate", startDate)
            .setParameter("endDate", endDate)
            .getResultList();
    }
```

```java
    /**
     * Get trade statistics for a symbol
     */
    public TradeStats getTradeStats(String symbol) {
        List<Object[]> result = entityManager.createQuery(
            "SELECT COUNT(t), SUM(t.quantity), AVG(t.price),
MIN(t.price), MAX(t.price) " +
            "FROM TradeEntity t WHERE t.symbol = :symbol AND
t.tradeDate = :today", Object[].class)
            .setParameter("symbol", symbol)
            .setParameter("today", LocalDate.now().format(DATE_FORMAT))
            .getResultList();

        TradeStats stats = new TradeStats();
        stats.symbol = symbol;

        if (!result.isEmpty() && result.get(0) != null) {
            Object[] row = result.get(0);
            stats.tradeCount = row[0] != null ? ((Number)
row[0]).longValue() : 0;
            stats.totalVolume = row[1] != null ? ((Number)
row[1]).longValue() : 0;
            stats.avgPrice = row[2] != null ? ((Number)
row[2]).doubleValue() : 0;
            stats.lowPrice = row[3] != null ? ((Number)
row[3]).doubleValue() : 0;
            stats.highPrice = row[4] != null ? ((Number)
row[4]).doubleValue() : 0;
        }

        return stats;
    }

    /**
     * Register trade listener for real-time updates
     */
    public void addTradeListener(Consumer<TradeEntity> listener) {
        tradeListeners.add(listener);
    }

    /**
     * Remove trade listener
```

```
     */
    public void removeTradeListener(Consumer<TradeEntity> listener) {
        tradeListeners.remove(listener);
    }

    private void notifyTradeListeners(TradeEntity trade) {
        for (Consumer<TradeEntity> listener : tradeListeners) {
            try {
                listener.accept(trade);
            } catch (Exception e) {
                LOG.error("Error notifying trade listener", e);
            }
        }
    }

    public static class TradeStats {
        public String symbol;
        public long tradeCount;
        public long totalVolume;
        public double avgPrice;
        public double lowPrice;
        public double highPrice;
        public double vwap;
    }
}
```

---

3. Trade-Driven Pricing Service

Purpose

This service implements real-time options pricing driven by trade execution events:

- **Consumes TradeExecuted events**: Registered as a listener to TradeService
- **Recomputes option prices**: Uses Black-Scholes model dynamically based on underlying price
- **Throttles updates**: Aggregates updates over 100ms to reduce UI churn
- **Publishes to UI**: Sends OptionPriceUpdated events via WebSocket to pricing consumers (options trading desks, monitoring systems)

Key Features
- **Event-Driven Pricing**: Listens to `onTradeExecuted()` real-time
- **Volatility Smile**: Estimates IV based on moneyness (OTM options have higher vol)
- **Throttled Broadcasting**: 100ms aggregation window before broadcasting to WebSocket
- **Multi-expiry/Strike**: Supports 9 strikes and 5 expiration dates per underlying

Code

```java
package com.helesto.service;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;
import java.util.function.Consumer;

import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import javax.enterprise.context.ApplicationScoped;
import javax.inject.Inject;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.helesto.model.TradeEntity;
import com.helesto.socket.WebSocketAggregator;

/**
 * G3-M4: Trade-Driven Pricing Updates
 * - Consumes TradeExecuted events and recomputes option fair prices dynamically
 * - Throttles/aggregates updates per symbol per 100ms to reduce UI churn
```

```
 * - Publishes OptionPriceUpdated events on WebSocket stream
 */
@ApplicationScoped
public class TradeDrivenPricingService {

    private static final Logger LOG =
LoggerFactory.getLogger(TradeDrivenPricingService.class);

    // Throttling configuration
    private static final long THROTTLE_INTERVAL_MS = 100; // 100ms
aggregation window
    private static final int MAX_PENDING_UPDATES = 1000;

    @Inject
    BlackScholesPricingService pricingService;

    @Inject
    ReferenceDataService referenceDataService;

    @Inject
    TradeService tradeService;

    @Inject
    WebSocketAggregator webSocketAggregator;

    @Inject
    MarketDataPoller marketDataPoller;

    // Latest trade prices by symbol
    private final ConcurrentMap<String, TradePrice> latestPrices = new
ConcurrentHashMap<>();

    // Pending price updates (throttled)
    private final ConcurrentMap<String, OptionPriceUpdate>
pendingUpdates = new ConcurrentHashMap<>();

    // Option chain configurations per underlying
    private final ConcurrentMap<String, OptionChainConfig> optionChains
= new ConcurrentHashMap<>();

    // Price update listeners
    private final List<Consumer<OptionPriceUpdate>> priceListeners =
new CopyOnWriteArrayList<>();
```

```java
    // Scheduler for throttled broadcasting
    private ScheduledExecutorService scheduler;

    @PostConstruct
    void init() {
        LOG.info("Initializing Trade-Driven Pricing Service...");

        // Initialize default option chains for major underlyings
        initializeOptionChains();

        // Register as trade listener
        tradeService.addTradeListener(this::onTradeExecuted);

        // Start throttled broadcast scheduler
        scheduler = Executors.newSingleThreadScheduledExecutor(r -> {
            Thread t = new Thread(r, "pricing-broadcaster");
            t.setDaemon(true);
            return t;
        });

        scheduler.scheduleAtFixedRate(this::broadcastPendingUpdates,
            THROTTLE_INTERVAL_MS, THROTTLE_INTERVAL_MS,
TimeUnit.MILLISECONDS);

        LOG.info("Trade-Driven Pricing Service initialized");
    }

    @PreDestroy
    void cleanup() {
        if (scheduler != null) {
            scheduler.shutdown();
        }
    }

    private void initializeOptionChains() {
        // Configure option chains for major symbols
        String[] majorSymbols = {"AAPL", "MSFT", "GOOGL", "AMZN",
"META", "NVDA", "TSLA"};

        for (String symbol : majorSymbols) {
            OptionChainConfig config = new OptionChainConfig();
            config.underlying = symbol;
```

```
            config.strikes = generateStrikes(symbol);
            config.expirations = generateExpirations();
            config.riskFreeRate = 0.05; // 5%
            config.baseVolatility = 0.25; // 25%
            optionChains.put(symbol, config);
        }

        LOG.info("Initialized option chains for {} underlyings",
majorSymbols.length);
    }

    private double[] generateStrikes(String symbol) {
        // Get current price from reference data
        ReferenceDataService.MarketData md =
referenceDataService.getMarketData(symbol);
        double basePrice = md != null ? md.lastPrice : 100.0;

        // Generate strikes around current price (-20% to +20%)
        double[] strikes = new double[9];
        for (int i = 0; i < 9; i++) {
            double offset = (i - 4) * 0.05; // -20%, -15%, -10%, -5%,
ATM, +5%, +10%, +15%, +20%
            strikes[i] = Math.round(basePrice * (1 + offset) * 100) /
100.0;
        }
        return strikes;
    }

    private double[] generateExpirations() {
        // Generate expirations: 7d, 14d, 30d, 60d, 90d
        return new double[] {7/365.0, 14/365.0, 30/365.0, 60/365.0,
90/365.0};
    }

    /**
     * Trade event handler - called when a trade executes
     */
    public void onTradeExecuted(TradeEntity trade) {
        long startTime = System.nanoTime();

        try {
            String symbol = trade.getSymbol();
            double tradePrice = trade.getPrice();
```

```
            long tradeQty = trade.getQuantity();

            // Update latest price for symbol
            TradePrice current = latestPrices.compute(symbol, (k, v) ->
{
                if (v == null) {
                    v = new TradePrice();
                    v.symbol = symbol;
                }
                // VWAP-style update
                v.totalVolume += tradeQty;
                v.volumeWeightedSum += tradePrice * tradeQty;
                v.lastPrice = tradePrice;
                v.lastUpdateTime = System.currentTimeMillis();
                v.tradeCount++;
                return v;
            });

            // Update reference data with latest price
            referenceDataService.updateLastPrice(symbol, tradePrice);

            // Update market data poller (triggers WebSocket broadcast)
            marketDataPoller.updatePriceFromTrade(symbol, tradePrice,
tradeQty);

            // Recompute option prices if symbol has options
            if (optionChains.containsKey(symbol)) {
                recomputeOptionPrices(symbol, tradePrice);
            }

        } catch (Exception e) {
            LOG.error("Error processing trade for pricing: {}",
e.getMessage());
        }
    }

    /**
     * Recompute all option prices for a symbol
     */
    private void recomputeOptionPrices(String symbol, double
underlyingPrice) {
        OptionChainConfig config = optionChains.get(symbol);
        if (config == null) return;
```

```
        // Recalculate strikes based on new price
        config.strikes = generateStrikes(symbol);

        List<OptionPrice> updatedPrices = new ArrayList<>();

        for (double strike : config.strikes) {
            for (double expiry : config.expirations) {
                // Calculate implied volatility from market conditions
                double vol = estimateVolatility(symbol, strike,
underlyingPrice, config.baseVolatility);

                // Price call
                double callPrice = pricingService.callPrice(
                    underlyingPrice, strike, expiry,
config.riskFreeRate, vol);
                BlackScholesPricingService.Greeks callGreeks =
pricingService.callGreeks(
                    underlyingPrice, strike, expiry,
config.riskFreeRate, vol);

                OptionPrice call = new OptionPrice();
                call.underlying = symbol;
                call.optionType = "CALL";
                call.strike = strike;
                call.expiry = expiry;
                call.price = callPrice;
                call.delta = callGreeks.delta;
                call.gamma = callGreeks.gamma;
                call.theta = callGreeks.theta;
                call.vega = callGreeks.vega;
                call.impliedVol = vol;
                call.underlyingPrice = underlyingPrice;
                updatedPrices.add(call);

                // Price put
                double putPrice = pricingService.putPrice(
                    underlyingPrice, strike, expiry,
config.riskFreeRate, vol);
                BlackScholesPricingService.Greeks putGreeks =
pricingService.putGreeks(
                    underlyingPrice, strike, expiry,
config.riskFreeRate, vol);
```

```
                OptionPrice put = new OptionPrice();
                put.underlying = symbol;
                put.optionType = "PUT";
                put.strike = strike;
                put.expiry = expiry;
                put.price = putPrice;
                put.delta = putGreeks.delta;
                put.gamma = putGreeks.gamma;
                put.theta = putGreeks.theta;
                put.vega = putGreeks.vega;
                put.impliedVol = vol;
                put.underlyingPrice = underlyingPrice;
                updatedPrices.add(put);
            }
        }

        // Queue update for throttled broadcast
        OptionPriceUpdate update = new OptionPriceUpdate();
        update.symbol = symbol;
        update.underlyingPrice = underlyingPrice;
        update.prices = updatedPrices;
        update.timestamp = System.currentTimeMillis();

        pendingUpdates.put(symbol, update);
    }

    /**
     * Estimate volatility based on price movement and moneyness
     */
    private double estimateVolatility(String symbol, double strike,
double spot, double baseVol) {
        // Volatility smile approximation
        double moneyness = strike / spot;
        double skew = 0;

        if (moneyness < 0.95) {
            // OTM puts - higher vol
            skew = (0.95 - moneyness) * 0.5;
        } else if (moneyness > 1.05) {
            // OTM calls - slightly higher vol
            skew = (moneyness - 1.05) * 0.3;
        }
```

```
        return baseVol + skew;
    }

    /**
     * Broadcast pending updates (called by scheduler)
     */
    private void broadcastPendingUpdates() {
        if (pendingUpdates.isEmpty()) return;

        // Get and clear pending updates
        Map<String, OptionPriceUpdate> updates = new
HashMap<>(pendingUpdates);
        pendingUpdates.clear();

        for (OptionPriceUpdate update : updates.values()) {
            try {
                // Broadcast to WebSocket
                broadcastOptionPriceUpdate(update);

                // Notify listeners
                for (Consumer<OptionPriceUpdate> listener :
priceListeners) {
                    listener.accept(update);
                }

            } catch (Exception e) {
                LOG.error("Error broadcasting option price update: {}",
e.getMessage());
            }
        }
    }
}
```

---

4. Telemetry Service - Observability

Purpose
Central observability service that tracks:
- **WebSocket metrics**: Connection count, messages in/out, broadcasts
- **Trade metrics**: Execution counts and timing

- **FIX protocol metrics**: Message lag, rejection rates
- **Order processing**: Fill counts, cancellations, rejections
- **Matching engine**: Match success rates, timing

Key Features
- **Thread-safe counters**: Uses `LongAdder` and `AtomicLong` for lock-free updates
- **Per-symbol tracking**: Tracks update frequencies by symbol
- **Timing measurements**: Captures nanosecond-precision timings for latency analysis
- **Aggregated health**: Single source of truth for monitoring and alerting

Code

```java
package com.helesto.service;

import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.LongAdder;

import javax.annotation.PostConstruct;
import javax.enterprise.context.ApplicationScoped;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

/**
 * G3-M6: Observability & Telemetry Service
 * - Message lag tracking
 * - Update frequency monitoring
 * - Compute time measurements
 * - Health metrics aggregation
 * - REST endpoint exposure for monitoring
 */
@ApplicationScoped
public class TelemetryService {

    private static final Logger LOG =
LoggerFactory.getLogger(TelemetryService.class);
```

```java
    // ================== FIX Message Metrics ==================
    private final LongAdder fixMessagesReceived = new LongAdder();
    private final LongAdder fixMessagesSent = new LongAdder();
    private final LongAdder fixMessagesRejected = new LongAdder();
    private final AtomicLong lastFixMessageTimestamp = new
AtomicLong(0);
    private final AtomicLong fixMessageLagNanos = new AtomicLong(0);

    // ================== Order Processing Metrics ==================
    private final LongAdder ordersReceived = new LongAdder();
    private final LongAdder ordersProcessed = new LongAdder();
    private final LongAdder ordersFilled = new LongAdder();
    private final LongAdder ordersCancelled = new LongAdder();
    private final LongAdder ordersRejected = new LongAdder();
    private final AtomicLong orderProcessingTimeNanos = new
AtomicLong(0);
    private final AtomicLong orderProcessingCount = new AtomicLong(0);

    // ================== Matching Engine Metrics ==================
    private final LongAdder matchAttempts = new LongAdder();
    private final LongAdder matchSuccesses = new LongAdder();
    private final LongAdder tradesGenerated = new LongAdder();
    private final AtomicLong matchingTimeNanos = new AtomicLong(0);
    private final AtomicLong matchingCount = new AtomicLong(0);

    // ================== Market Data Metrics ==================
    private final LongAdder marketDataUpdates = new LongAdder();
    private final LongAdder marketDataBroadcasts = new LongAdder();
    private final AtomicLong lastMarketDataTimestamp = new
AtomicLong(0);
    private final Map<String, AtomicLong> symbolUpdateCounts = new
ConcurrentHashMap<>();

    // ================== WebSocket Metrics ==================
    private final LongAdder wsMessagesIn = new LongAdder();
    private final LongAdder wsMessagesOut = new LongAdder();
    private final AtomicLong activeWsConnections = new AtomicLong(0);
    private final LongAdder wsBroadcasts = new LongAdder();

    // ================== Options Pricing Metrics ==================
    private final LongAdder optionPricesComputed = new LongAdder();
```

```java
    private final AtomicLong pricingComputeTimeNanos = new
AtomicLong(0);
    private final AtomicLong pricingComputeCount = new AtomicLong(0);

    // ================== System Health ==================
    private final AtomicLong startTimeMs = new AtomicLong(0);
    private final LongAdder errors = new LongAdder();
    private final LongAdder warnings = new LongAdder();

    @PostConstruct
    void init() {
        startTimeMs.set(System.currentTimeMillis());
        LOG.info("TelemetryService initialized");
    }

    // ================== Recording Methods ==================

    public void recordFixMessageReceived() {
        fixMessagesReceived.increment();
        lastFixMessageTimestamp.set(System.currentTimeMillis());
    }

    public void recordFixMessageSent() {
        fixMessagesSent.increment();
    }

    public void recordFixMessageRejected() {
        fixMessagesRejected.increment();
    }

    public void recordFixMessageLag(long lagNanos) {
        fixMessageLagNanos.set(lagNanos);
    }

    public void recordOrderReceived() {
        ordersReceived.increment();
    }

    public void recordOrderProcessed(long processingTimeNanos) {
        ordersProcessed.increment();
        orderProcessingTimeNanos.addAndGet(processingTimeNanos);
        orderProcessingCount.incrementAndGet();
    }
```

```java
    public void recordOrderProcessed() {
        ordersProcessed.increment();
    }

    public void recordOrderFilled() {
        ordersFilled.increment();
    }

    public void recordOrderCancelled() {
        ordersCancelled.increment();
    }

    public void recordOrderRejected() {
        ordersRejected.increment();
    }

    public void recordMatchAttempt(boolean success, long
matchTimeNanos) {
        matchAttempts.increment();
        if (success) {
            matchSuccesses.increment();
        }
        matchingTimeNanos.addAndGet(matchTimeNanos);
        matchingCount.incrementAndGet();
    }

    public void recordTradeGenerated() {
        tradesGenerated.increment();
    }

    public void recordMarketDataUpdate(String symbol) {
        marketDataUpdates.increment();
        lastMarketDataTimestamp.set(System.currentTimeMillis());
        symbolUpdateCounts.computeIfAbsent(symbol, k -> new
AtomicLong(0)).incrementAndGet();
    }

    public void recordMarketDataBroadcast() {
        marketDataBroadcasts.increment();
    }

    public void recordWsMessageIn() {
```

```
        wsMessagesIn.increment();
    }

    public void recordWsMessageOut() {
        wsMessagesOut.increment();
    }

    public void recordWsConnection(boolean connected) {
        if (connected) {
            activeWsConnections.incrementAndGet();
        } else {
            activeWsConnections.decrementAndGet();
        }
    }

    public void recordWsBroadcast() {
        wsBroadcasts.increment();
    }

    public void recordOptionPriceComputed() {
        optionPricesComputed.increment();
    }

    public void recordError() {
        errors.increment();
    }
}
```

---

5. Frontend Market Data Service

Purpose
Angular service that handles real-time market data updates from the backend:
- Establishes WebSocket connection to `/ws/aggregator`
- Maintains BehaviorSubjects for reactive updates
- Fallback to polling if WebSocket fails
- Provides observables for components to subscribe to market data changes

Key Features

- **Dual communication**: WebSocket for real-time + HTTP polling fallback
- **Data caching**: Uses BehaviorSubject to cache latest state
- **Auto-reconnect**: Retries WebSocket connection after 5s
- **Type-safe interfaces**: Security, MarketData, OptionPrice interfaces

Code

```typescript
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, Subject, BehaviorSubject, interval } from 'rxjs';
import { map, catchError, switchMap, startWith } from 'rxjs/operators';

export interface Security {
  symbol: string;
  name: string;
  sector: string;
  securityType: string;
  currency: string;
  lotSize: number;
  tickSize: number;
  maxOrderSize: number;
  tradeable: boolean;
}

export interface MarketData {
  symbol: string;
  lastPrice: number;
  bid: number;
  ask: number;
  bidSize: number;
  askSize: number;
  volume: number;
  open: number;
  high: number;
  low: number;
  change: number;
  changePercent: number;
  timestamp: string;
}
```

```
export interface OptionPrice {
  symbol: string;
  price: number;
  delta: number;
  gamma: number;
  theta: number;
  vega: number;
  rho: number;
  impliedVol: number;
  intrinsicValue: number;
  timeValue: number;
}

@Injectable({
  providedIn: 'root'
})
export class MarketDataService {
  private marketDataSubject = new BehaviorSubject<Map<string,
MarketData>>(new Map());
  private securitiesSubject = new BehaviorSubject<Security[]>([]);
  private wsConnection: WebSocket | null = null;

  marketData$ = this.marketDataSubject.asObservable();
  securities$ = this.securitiesSubject.asObservable();

  constructor(private http: HttpClient) {
    this.loadSecurities();
    this.startPolling();
    this.connectWebSocket();
  }

  private loadSecurities(): void {
    this.http.get<Security[]>('/api/securities').subscribe({
      next: (securities) => this.securitiesSubject.next(securities),
      error: (err) => console.error('Failed to load securities', err)
    });
  }

  private startPolling(): void {
    interval(2000).pipe(
      startWith(0),
      switchMap(() => this.http.get<MarketData[]>('/api/marketdata'))
    ).subscribe({
```

```
      next: (data) => {
        const map = new Map<string, MarketData>();
        data.forEach(md => map.set(md.symbol, md));
        this.marketDataSubject.next(map);
      },
      error: (err) => console.error('Market data poll failed', err)
    });
  }

  private connectWebSocket(): void {
    try {
      const protocol = window.location.protocol === 'https:' ? 'wss:' :
'ws:';
      const host = window.location.hostname;
      this.wsConnection = new
WebSocket(`${protocol}//${host}:8090/ws/aggregator`);

      this.wsConnection.onmessage = (event) => {
        try {
          const update = JSON.parse(event.data);
          if (update.type === 'marketData' || update.symbol) {
            const currentData = this.marketDataSubject.value;
            currentData.set(update.symbol, update);
            this.marketDataSubject.next(new Map(currentData));
          }
        } catch (e) {
          // Ignore parse errors
        }
      };

      this.wsConnection.onerror = () => {
        console.log('WebSocket connection failed, using polling');
      };

      this.wsConnection.onclose = () => {
        setTimeout(() => this.connectWebSocket(), 5000);
      };
    } catch (e) {
      console.log('WebSocket not available, using polling');
    }
  }

  getSecurities(): Observable<Security[]> {
```

```
    return this.http.get<Security[]>('/api/securities');
  }

  getAllMarketData(): Observable<MarketData[]> {
    return this.http.get<MarketData[]>('/api/marketdata');
  }

  getMarketDataForSymbol(symbol: string): Observable<MarketData> {
    return this.http.get<MarketData>(`/api/marketdata/${symbol}`);
  }

  getOptionPrice(symbol: string, strike: number, expiry: string,
isCall: boolean): Observable<OptionPrice> {
    const optionType = isCall ? 'CALL' : 'PUT';
    return
this.http.get<OptionPrice>(`/api/options/price?symbol=${symbol}&strike=
${strike}&expiry=${expiry}&optionType=${optionType}`);
  }
}
```

---

6. Frontend Order Service

Purpose
Angular service that manages order operations and real-time order updates:
- Establishes WebSocket connection to `/socket/order`
- Emits order updates via Subject
- Provides REST endpoints for CRUD operations
- Auto-reconnect on WebSocket close

Key Features
- **Real-time Order Updates**: WebSocket messages emitted to `orderUpdates$()` Subject
- **Auto-reconnect**: 3-second retry interval
- **Trade Retrieval**: HTTP endpoint to fetch executed trades
- **Session Management**: Gets session status and events

Code

```typescript
```

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, Subject } from 'rxjs';
import { Order, Trade, SessionStatus, QuickFixResponseRequest,
QuickFixResponseResult, SessionEvent } from '../models/order.model';

@Injectable({
  providedIn: 'root'
})
export class OrderService {

  private apiUrl = '/api/orders';
  private orderUpdates = new Subject<Order>();

  constructor(private http: HttpClient) {
    this.initWebSocket();
  }

  getOrders(): Observable<Order[]> {
    return this.http.get<Order[]>(this.apiUrl);
  }

  getOrder(clOrdId: string): Observable<Order> {
    return this.http.get<Order>(`${this.apiUrl}/${clOrdId}`);
  }

  createOrder(order: Order): Observable<Order> {
    return this.http.post<Order>(this.apiUrl, order);
  }

  cancelOrder(clOrdId: string): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${clOrdId}`);
  }

  getTrades(): Observable<Trade[]> {
    return this.http.get<Trade[]>('/api/trades');
  }

  getSessionStatus(): Observable<SessionStatus> {
    return this.http.get<SessionStatus>('/api/session');
  }
```

```
  sendQuickFixResponse(request: QuickFixResponseRequest):
Observable<QuickFixResponseResult> {
    return
this.http.post<QuickFixResponseResult>('/api/session/respond',
request);
  }

  getSessionEvents(limit = 20): Observable<SessionEvent[]> {
    return
this.http.get<SessionEvent[]>(`/api/session/events?limit=${limit}`);
  }

  orderUpdates$(): Observable<Order> {
    return this.orderUpdates.asObservable();
  }

  private initWebSocket(): void {
    try {
      const protocol = window.location.protocol === 'https:' ? 'wss:' :
'ws:';
      const ws = new
WebSocket(`${protocol}//${window.location.host}/socket/order`);

      ws.onmessage = (event) => {
        const order = JSON.parse(event.data);
        this.orderUpdates.next(order);
      };

      ws.onerror = (error) => {
        console.error('WebSocket error:', error);
      };

      ws.onclose = () => {
        console.log('WebSocket closed, attempting to reconnect...');
        setTimeout(() => this.initWebSocket(), 3000);
      };
    } catch (error) {
      console.error('WebSocket initialization error:', error);
    }
  }
}
```

ARCHITECTURE SUMMARY

Event Flow

```
TradeExecuted (Order Match)
    ↓
TradeService.createTrade()
    ↓
notifyTradeListeners()
    ├→ TradeDrivenPricingService.onTradeExecuted()
    │    ├→ recomputeOptionPrices()
    │    ├→ pendingUpdates.put()
    │    └→ [100ms later] broadcastPendingUpdates()
    │        └→ WebSocketAggregator.broadcastOptionPriceUpdate()
    │
    ├→ PositionTrackingService.onTrade()
    │    └→ updatePosition()
    │
    └→ [Audit Trail] AuditTrailService.writeTradeEvent()

WebSocketAggregator
    ├→ broadcastTrade() → All subscribed clients via TRADE message
    ├→ broadcastOptionsPrice() → OPTIONS_PRICE subscribers
    └→ [Reconnect] handleTradeReplay() → Send last 50 trades per symbol

UI (Frontend)
    ├→ MarketDataService
    │    └→ WebSocket /ws/aggregator
    │        ├→ Receive TRADE messages
    │        ├→ Receive OPTIONS_PRICE messages
    │        └→ auto-reconnect after 5s
    │
    └→ OrderService
        └→ WebSocket /socket/order
            ├→ Receive ORDER updates
            └→ auto-reconnect after 3s

Telemetry (Monitoring)
    ├→ recordWsConnection()
    ├→ recordWsMessageIn/Out()
    ├→ recordTradeGenerated()
```

```
    └→ recordOptionPriceComputed()
```

Key Design Patterns

1. **Observer Pattern**: TradeListener mechanism for event publishing
2. **Throttling Pattern**: 100ms aggregation window for option price updates
3. **Message Batching**: 50ms WebSocket message batching
4. **Graceful Degradation**: Fallback to HTTP polling if WebSocket fails
5. **Telemetry Integration**: Every critical operation recorded for observability

Report by:
Arkadeep Ganguly,
230958064